

前言

近些年来，随着前端项目复杂度的不断提高，**前端工程化** 被越来越多的公司与开发者所重视。

从业务层面来说 **前端工程化** 俨然已成为一线互联网前端团队的标配，稍具规模的团队都会根据自身业务设计出一套符合当前业务需求的前端架构。从开发者角度来说 **前端工程化** 已逐渐成为**普通前端**与**资深前端**的分界线。例如很多大厂的普通前端只负责编写业务代码，而资深前端则会使用 **前端工程化** 解决生产问题，以接触更高层次的 **前端架构设计**，向技术管理岗晋升。

这也是为何很多 **前端开发者** 已熟练使用 **react/vue/next/nuxt** 等前端框架，但升职加薪依然遥不可及，因此**前端开发者想要突破工作局限性，实现跳跃式涨薪，掌握前端工程化必不可少！**

概念：前端工程化是什么

说了这么多，**前端工程化** 到底是什么？**前端工程化**指使用软件工程的技术与方法对前端开发的技术、工具、流程、经验、方案等指标 **标准化**，它具备**模块化**、**组件化**、**规范化**、**自动化**四大特性，主要目的是 **降低成本** 与 **增加效率**。



很多同学在不了解 **前端工程化** 前，遇到以下情况经常不知所措。

- 构建配置、打包配置、公共组件、工具函数等代码片段，每次新开项目都要复制粘贴

- 团队成员的编码风格大相径庭，导致从仓库拉取下来的代码运行起来让控制台一片红
- 团队协作的规范、环境、模块、仓库和文档，太多基建措施导致团队新成员无从入手
- 随着需求迭代引起项目结构与工程文件不断变化，处理不当让项目直接走向重构道路

随着需求迭代的步伐加速，上述问题甚至更多问题无可避免地发生，若一开始未对项目做一些相关工程化措施，项目维度会变得凌乱不堪，甚至达到无法维护的可能，最终只会走向重构道路。

实际上只要将 前端工程化 的开发思维与解决方案应用到项目中，利用好它的优势，就能轻松实现这些非业务需求，为业务降本增效。

总之， 前端工程化 不是某个具体的工具，而是**对项目的整体架构与整体规划，使开发者能在未来可判时间内动态规划发展走向，以提升整个项目对用户的服务周期。**学习 前端工程化 不仅能理解清楚一个项目的完整流程，遇到困难也能在复杂的流程中快速定位并解决问题，还能根据自身知识储备制定一些可扩展流程，甚至可预见项目的未来发展方向。

课程：如何系统学习前端工程化

想要系统学习 前端工程化 不是一件容易的事情。看到这，有些同学可能会说：不就是封装组件库嘛，我也会 前端工程化 呀！

前端工程化 可不只是会封装组件库就行。

首先要有明确前后端任务分离的能力。简而言之，能一眼看出该任务属于前端还是后端，划分好前后端的职责更利于 前端工程化 的接入。这也是基于 前端工程化 解决问题的基础。

其次要掌握 前端工程化 的四大核心特性，**模块化、组件化、规范化和自动化**。知道它们如何实现，它们各自标准是什么，因为所有前端工作流程都离不开这些核心内容。

虽然很多社区也有一些 前端工程化 相关文章教程，但它们只针对 前端工程化 的某个技能，零零星星的知识点与断层式的学习很难充分实践 前端工程化 。



JowayYoung

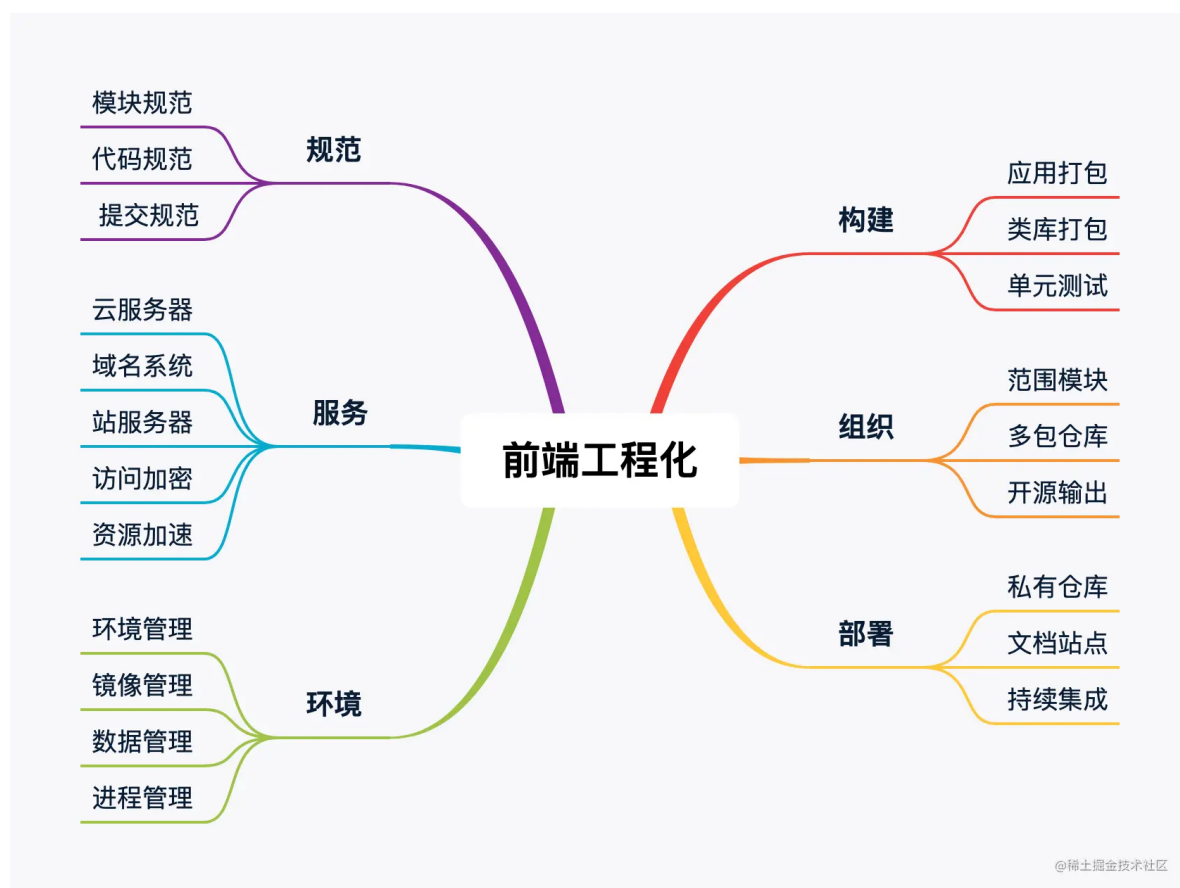
网易资深前端工程师，《玩转CSS的艺术之美》作者



@稀土掘金技术社区

作为一名 网易 的资深前端工程师，近三年来，我一直在 前端工程化 领域实践，利用工程架构的知识重构了众多项目，包括但不限于 脚手架、组件库、工具库、多包仓库、私有仓库、接口系统、文档系统、监控系统、CI/CD、可移植容器 等。有了这些 前端工程化 技术的加持，我负责的项目也从手动处理流程全部替换为自动处理流程，以解放团队双手，让其他成员更专注于自身业务需求。

基于此，我将这些经验总结下来，希望通过本课程与你分享。为了让 前端工程化 的课程适合更多开发者，我会以小白的身份，与你一起深入工程的各个环节，带领你走完 前端工程化 落地的全过程。



本课程主要分为**6大模块**。

✅ **规范篇**：熟悉 模块/代码/提交 三大开发阶段规范，通过规范约束自己，保障工作质量与提升开发效率

✅ **服务篇**：熟悉 云服务器/域名系统/站服务器 部署服务环境，掌握整体部署与工具配置，学会独立上线应用与服务

- ✓ **环境篇**：熟悉 **Node/Nvm/Npm** 部署开发环境，独立搭建一个 **接口服务**，实践 **环境/镜像/数据/进程** 四种 **Node** 应用方式
- ✓ **构建篇**：熟悉 **构建工具** 打包类库模块，独立封装一个 **类库模块**，结合 **测试用例** 保障代码的生产质量
- ✓ **组织篇**：熟悉 **Monorepo**模式 管理多包仓库，独立维护一个 **多包仓库**，结合 **Npm Scope** 发布模块到公共仓库
- ✓ **部署篇**：熟悉 **自动化工具** 部署前端项目，独立打造一个 **私有仓库** 与 **文档站点**，结合 **CI/CD** 在提交代码时自动部署到公网

细节：前端工程化的特性

对于 **前端工程化** 的四大特性，在进入学习前有必要了解它们各自的细节。

模块化

模块化指将一个复杂应用根据预设规范封装为多个块并组合起来，对内实现数据私有化，对外暴露接口与其它模块通信。

模块化 是 **前端工程化** 的重中之重。它在 **前端工程化** 中具体表现为：在文件层面上对代码与资源实现拆分与组装，将一个大文件拆分为互相依赖的小文件，再统一拼装与加载。

对于一个完善的 **Web项目**，一般是 **SPA/MPA**，推荐使用以下目录结构将整个项目划分为各种通用模块。为了让目录结构更突出其功能，就不包括那些杂七杂八的工具链配置文件了。

```
project
├─ dist          # 输出目录
│  └─ prod      # 生产环境执行代码
│  └─ test      # 测试环境执行代码
├─ src          # 源码目录
│  └─ apis      # 接口模块：包括全局接口请求的功能，控制数据定向转换
│  └─ assets    # 资源模块：包括样式、脚本、字体、图像、音频、视频等资源文件
│  └─ components # 组件模块：包括全局通用的基础组件、皮肤主题和字体图标
│  └─ layouts   # 布局模块：包括以布局为最小粒度的组件集合，由至少一个基础组件组成
│  └─ flows     # 流程模块：包括以流程为最小粒度的组件集合，由至少一个基础组件组成
│  └─ pages     # 页面模块：包括以页面为最小粒度的组件集合，由至少一个基础组件组成
│  └─ routes    # 路由模块：包括全局页面跳转的功能，控制页面自由切换
│  └─ stores    # 数据模块：包括全局数据状态的功能，控制数据驱动视图
│  └─ views     # 视图模块：包括以视图为最小粒度的组件集合，由至少一个基础组件组成
```

txt

```
| └─ utils          # 工具模块：包括全局通用的常量与方法
| └─ index.html     # 模板入口文件
| └─ index.js       # 脚本入口文件
| └─ index.scss     # 样式入口文件
└─ package.json
```

对于一个完善的 **Node项目**，一般是 **接口系统**，推荐使用以下目录结构将整个项目划分为各种通用模块。为了让目录结构更突出其功能，就不包括那些杂七杂八的工具链配置文件了。

```
project.txt
├─ dist          # 输出目录
| └─ prod        # 生产环境执行代码
| └─ test        # 测试环境执行代码
├─ src           # 源码目录
| └─ assets      # 资源模块：包括样式、脚本、字体、图像、音频、视频等资源文件
| └─ models      # 模型模块：包括全局数据模型的功能
| └─ routes      # 路由模块：包括全局接口请求的功能
| └─ utils       # 工具模块：包括全局通用的常量与方法
| └─ index.js    # 脚本入口文件
└─ package.json
```

当然这只是 **模块化** 的第一步，后续章节会有更多内容涉及 **模块化**。

组件化

组件化指将一个具备通用功能的交互设计划分为模板、样式和逻辑组成的功能单元，对内管理内部状态满足交互需求，对外提供属性接口扩展用户需求。

组件化是 **前端工程化** 的重要基础。它实现了代码更高层次的复用性，提升开发效率。组件的封装也是对象的封装，同样要做到**高内聚低耦合**，**组件化**的项目不仅利于**单元测试**的进行，同样也利于需求迭代的推进。

优秀的 **组件化** 遵循以下设计哲学。

- 将设计图划分为最小组件层级
- 使用预设规范创建组件静态版本
- 确定组件内部最小且完整的状态的表示方式
- 确定组件内部最小且完整的状态的存放方式
- 实现数据流的正向传递与反向传递

有些同学可能会将 **模块化** 与 **组件化** 混淆，其实了解它们的概念就很易区分了。**模块化**着重在文件层面上对代码与资源实现拆分与组装，**组件化**着重在功能层面上对交互与设计实现拆分与组装。

规范化

规范化指将一系列预设规范接入工程各个阶段，通过各项指标标准化开发者的工作流程，引导开发者在团队协作中往更好的方向发展。

规范化是 **前端工程化** 的重要部分。它有效地将一盘松散的规范通过指定标准凝聚在一起，将所有工作流程标准化，协同所有开发者以标准化的方式定义工作流程，同时也影响着代码、文档和日志，甚至影响着每个开发者及其团队发展方向，因此每个成熟的前端团队都有一套身经百战的 **规范化方案**。

规范化更多应用在团队协作中，为每个开发者指明一个方向，引领着成员往该方向走。若团队无 **规范化**，每个开发者各做各的事情，在合并代码时肯定会发生争吵，甚至影响工作效率。

自动化

自动化指将一系列繁琐重复的工作流程交由程序根据预设脚本自动处理，整个工作流程无需人工参与，以解放开发者双手让其更专注业务需求的开发。

自动化是 **前端工程化** 的智能部分。它既可解放双手又能节省大量时间做更多有意义的事情，常见 **自动化** 场景包括但不限于 **自动化构建**、**自动化测试**、**自动化打包**、**自动化发布** 和 **自动化部署**，更高级的自动化场景包括但不限于 **持续集成**、**持续交付** 和 **持续部署**。以 **自动化构建** 为例，又可将其划分为以下子任务，这些子任务分布在 **自动化构建** 不同阶段，在不同阶段的最佳时刻会调用相关工具处理相关流程。



任务	职责
Stylelint	校验样式代码
Eslint	校验脚本代码
Postcss	Postcss → CSS
Sass	SASS → CSS
Less	LESS → CSS
Babel	ES6 → ES5
TypeScript	TS → JS

自动化 整体重心偏向于构建，构建为工程服务，工程又为用户服务，因此一个项目会演化出至少两种运行环境，分别是 **开发环境** 与 **生产环境**。其中开发环境工程为开发者服务，生产环境工程为用户服务。

学习 **前端工程化** 能让一个开发者在任何时刻面对任何情况都能使用工程化的思想解决问题。掌握 **前端工程化** 可通过以下方面思考，相信 **前端工程化** 对工作能力与岗位竞争都会有很大帮助。

- **前后分离**：前端应自成体系且与后端分离，包括但不限于规范、服务、环境、构建、组织和部署方面
- **技术选型**：不能以一个框架满足所有业务场景，需制定多套框架解决方案避免技术瓶颈的出现
- **重构封装**：新生技术不断涌现就要避免改头换面式的重构，重复需求不断出现就要学会举一反三的封装
- **工程设计**：解决方案要合理分层且互相独立，随时应对各种变化，任何一层可低成本被替换与淘汰

课前准备：万事俱备只欠东风



前端工程化 从名字上来看就能想象出其与配置或流程有着众多关联，因此学习 前端工程化 可能与平时学习 `react/vue` 等框架编程有着不一样的体验。这些体验无法像框架编程那样使用常见套路或技巧去编码，而是得清楚当前流程的所有步骤以及使用何种工具互相搭配与配置完成针对流程的处理。

前端工程化 涉及众多的环境与工具，每个流程涉及至少一种环境或一种工具。在学习本课程时会用到很多与 前端工程化 相关环境与工具，我提前将它们罗列出来，好让你有心理准备。

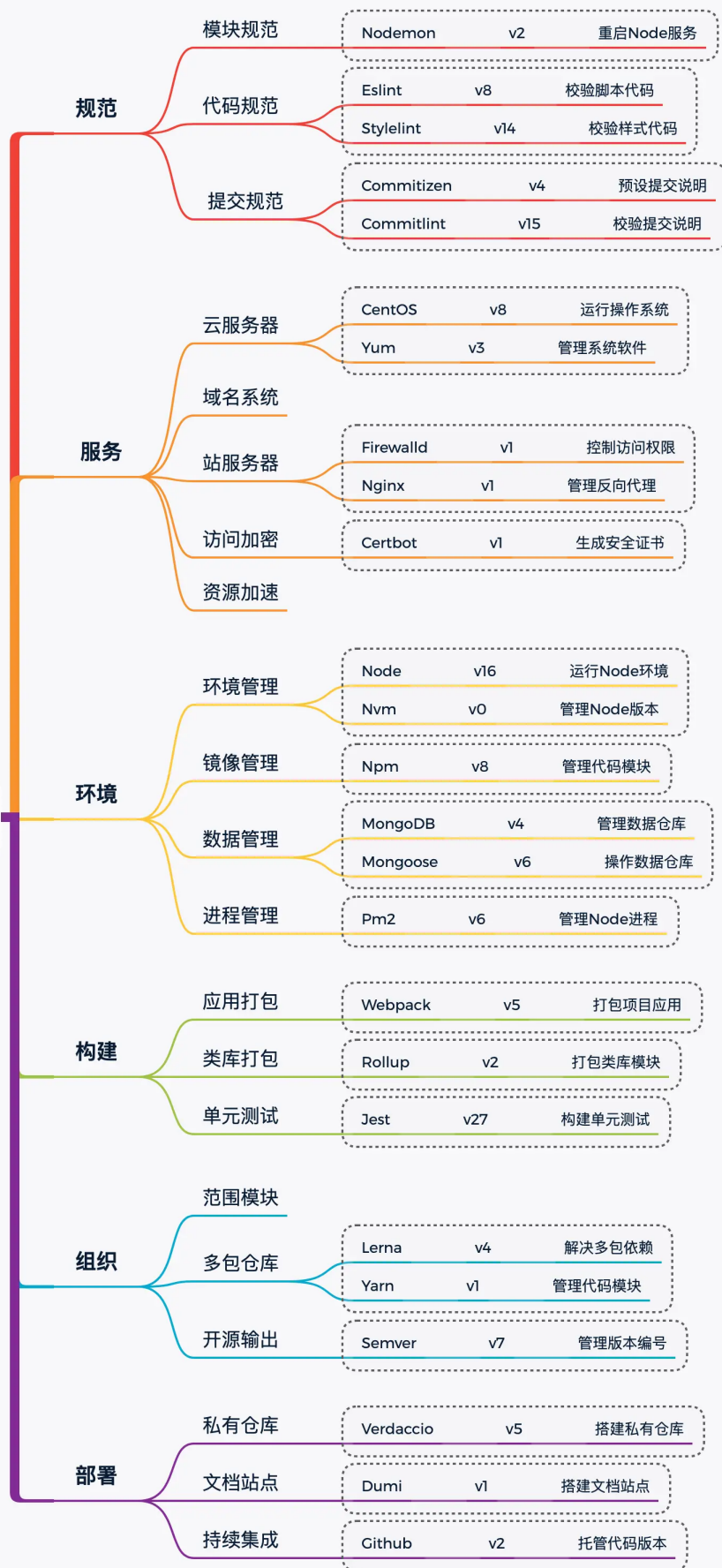
为了方便对以下环境与工具有一个第一印象，都使用 六字真言 概括它们的主要功能，不要太在意是否准确，因为后续章节会详细讲述这些环境与工具。

没错，总结大师就是喜欢总结！

txt



环境与工具



@稀土掘金技术社区

很多同学在一线开发岗位工作多年后才能体会到 **前端工程化** 的重要性。这些知识因为理论性太强，一般很少有人做总结与分享，基本都是只可意会不可言传的内容，而这些内容的积累也是从一个 **Developer** 走向 **Team Leader** 的关键。我花了 **1年** 时间写下本课程，也是为了将这些只可意会不可言传的内容转换为都可言传的内容，帮助更多开发者突破自己，完成职业生涯的转变。

总之，在学习具体技能的同时要重视工程化的解决思路，灵活应用它对自身的工作能力与竞争力都会有很大的提升。

最后，本课程囊括的 **前端工程化** 应用场景都很常见。若后续有时间，我也会尽力完善更多内容，若你有更好的建议也欢迎加群与我沟通。期待与你同行，一起落地 **前端工程化**，告别纸上谈兵！

消化下，下章正式进入 **前端工程化** 的学习之旅。最后记住这句话：**无论处于何种阶段，深入了解前端工程化都是很有必要的！！**

留言

输入评论 (Enter换行, Ctrl + Enter发送)

发表评论

全部评论 (47)



qingkooo

前端开发 1月前

无论处于何种阶段，深入了解前端工程化都是很有必要的

点赞 回复



空城以北

web前端工程师 @ 百腾... 1月前

打卡。大公司没用 小公司用不到系列

1 1

JowayYoung (作者) 1月前

前端工程化不是买个课程或者随便找几篇文章就会了，需要积累，积累，积累。。。



👍 点赞 💬 回复



嘻哈同学97986 web前端 1月前

打卡

👍 点赞 💬 回复



木由金 2月前



👍 点赞 💬 回复



\ Bo 2月前



👍 点赞 💬 回复



一名菜鸟极客 全栈开发 @ 墨子文化 2月前

冲冲冲

👍 点赞 💬 回复



Geek喜多川 码字员 @ 能力有限公司 2月前



👍 点赞 💬 回复



静听花亦落 老年前端 @ 阿巴阿巴 2月前

脑图上的v1 v2 v3是什么意思 🤔

👍 点赞 💬 2

JowayYoung (作者) 2月前

版本呀

👍 点赞 💬 回复



静听花亦落 回复 JowayYoung 2月前

哦，懂了，哈哈

“版本呀”

👍 点赞 💬 回复



codeLife 前端开发工程师 @ 啥... 2月前

请教一下，flows流程模块中是做什么的？



👍 点赞 🗨️ 2

JowayYoung 🏆 (作者) 2月前

通常是一些弹窗类型的业务模块

👍 点赞 🗨️ 回复



codeLife 回复 JowayYoung 2月前

好的，谢谢指导

“通常是一些弹窗类型的业务模块”

👍 点赞 🗨️ 回复



浊清 JY.3 2月前

请教一下，views视图模块 与 pages页面模块的区别是？

👍 点赞 🗨️ 1

JowayYoung 🏆 (作者) 2月前

视图与页面的区别：一个页面对应一个URL，但是可以在一个页面内多换多个视图，而还是同一个URL，所以一个页面可包含多个视图，一个页面只对应一个URL，但多个视图可对应一个URL，这些视图只能通过页面的不同状态展示或隐藏

👍 点赞 🗨️ 回复



小孩摸鸡嘎 JY.3 🏆 @ 3月前

打卡

👍 点赞 🗨️ 回复



用户6531906159... JY.1 3月前

多发点肥嘟嘟

👍 点赞 🗨️ 1

JowayYoung 🏆 (作者) 2月前

谁是肥嘟嘟 🙋

👍 点赞 🗨️ 回复



EEEEEEEEEE JY.4 前端 3月前

打卡

👍 点赞 🗨️ 回复

哆啦A梦控余 JY.3 3日前



后面会出大前端全链路监控系统的开发搭建教程吗?

👍 点赞 💬 1

JowayYoung (作者) 3月前

可以考虑下，这个搭建比较繁琐，需要想下怎样写才行，大工作量

👍 1 💬 回复



WangShuXian6 4月前

可否提供一些流程模块 的示例代码? 想学习一下这个

👍 1 💬 1

JowayYoung (作者) 3月前

具体是什么，可以描述下吗

👍 点赞 💬 回复



LCHub低代码社区 项目经理 @ LCHub 4月前

不错

👍 点赞 💬 回复



啊Ben学前端 前端 @ 字节跳动 4月前

打卡第一天

👍 点赞 💬 回复



很帅很愁人 全村工程师 @ 下王庄... 4月前

确实总结大师，佩服!

👍 点赞 💬 1

JowayYoung (作者) 4月前

谢谢支持 ❤️

👍 1 💬 回复



前端老码农 4月前

确实总结大师，佩服!

👍 1 💬 1

JowayYoung (作者) 4月前

谢谢支持 ❤️

👍 点赞 💬 回复



瓦嘞嘞 

4月前

打卡打卡

 点赞  回复

查看全部 47 条回复 

